Using Machine Learning To Predict Student Performance

Source: *Guilford College*

Contributed by: Dusenge, Yves

Stable URL: https://www.jstor.org/stable/community.20472372

Rights Notes: Theses may not be reproduced (either in print or online) without prior written permission of the author. Proper citation guidelines should be used when quoting or paraphrasing from a thesis. Thesis work remains the intellectually property of the student author unless specified otherwise and is provided for educational use only.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

This item is being shared by an institution as part of a Community Collection.
For terms of use, please refer to our Terms & Conditions at https://about.jstor.org/terms/#whats-in-jstor

Guilford College is collaborating with JSTOR to digitize, preserve and extend access to *Guilford College*

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

USING MACHINE LEARNING TO PREDICT STUDENT PERFORMANCE

YVES DUSENGE

A thesis submitted to the faculty of Guilford College

in partial fulfillment of the requirements for

The Honors Program

IN COMPUTING TECHNOLOGY INFORMATION SYTEMS

May 11, 2017

Committee:

_____

DR. RICHARD SCHILHAVY, Chair

_____

DR. NATALYA SHELKOVA

_____

DR. DANIELLE MORAN

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Abstract

The aim of this thesis is to develop and test different machine learning techniques using the dataset on Student Academic Performance that was posted on Kaggle. Kaggle is a platform for predictive modeling on which individuals and companies post their data, and data analysts from all over the world compete and collaborate to produce the best models. Using Python, different machine learning algorithms will be applied to analyze the data, including logistic regression, random forest and support vector machines classifier. After the algorithms have been applied and the algorithm with the greatest performance identified, a web application will be built to display the results of the data analyzed.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Acknowledgments

I would like to thank the faculty in the Computing Technology Information Systems department at Guilford College for providing me the opportunity for an excellent educational growth experience. I would especially thank Dr. Richard Schilhavy for his patience, mentoring and support throughout this thesis. I would also extend my appreciation to Dr. Natalya Shelkova for her encouragement, mentoring throughout my time at Guilford college, Dr. Danielle Moran for her assistance in this process and support, Dr. Heather Hayton for continuous support and positive encouragement in my growth as a student in the honors program.

Lastly, I thank my family for their sacrifice and support to pursue my goals. Without their continued encouragement, completion of this thesis may not have been possible.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Contents

1. Introduction
   1.1 Objectives
2. Background
   2.1 Schools and Student Performance
   2.2 Dataset
3. Technologies/Algorithms Used
   3.1 Python
   3.2 Jupyter Notebook
   3.3 Flask- A Python Micro Web Framework
   3.4 Machine Learning Packages
   3.5 Categories of Machine Learning Algorithms
4. A Framework for Machine Learning in Predictive Modeling
   4.1 Exploring and Preprocessing Data
   4.2 Building Models
5. Classification Methods
   5.1 Logistic Regression
   5.2 Support Vector Machines
   5.3 Random Forrest
   5.4 Summary
6. In-depth analysis
   6.1 Data visualization and preprocessing
   6.2 Interpretation and Evaluation
7. Web application
8. Summary and Future enhancements
9. References
10. Appendix A: Visualizations code
11. Appendix B: Predictive Models Building code
12. Appendix C: Web Application code

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

List of Figures

Figure 3.5.1 Linear Regression Example……...12
Figure 3.5.2 Classification Example……..12
Figure 4.3.1 Example of a twofold cross validation……..14
Figure 4.3.2 Bias vs Variance……..……15
Figure 5.3.1 Logistic Regression Example. 16
Figure 5.3.2 Support Vector Machine Example 17
Figure 5.4.3 Decision Tree Example. 19
Figure 6.1.1. Non-Categorical Features visualization…….. 21
Figure 6.1.2. Correlation Map…….....22
Figure 6.1.3. Categorical Features visualization …….....23
Figure 6.2.1. Logistic Regression Confusion Matrix.....25
Figure 6.2.2 Support Vector Machine Confusion Matrix….....….....25
Figure 6.2.3 Decision Tree Confusion Matrix. 25
Figure 6.2.4 Random Forrest Confusion Matrix. 25
Figure 6.2.7 Feature Importance……...28
Figure 7.1 Web application input selection screen shot……....……29
Figure 7.2 Prediction result screen shot……29

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 1

Introduction

Access to education is thought of as providing people from different backgrounds the opportunity to learn, gain knowledge and grow as human beings. However, this process requires institutions to continue supporting the students it admits and enable the student to perform well. For this project, success will be defined as the capability of a school to predict a student performance. When a student performs well in class and is getting high grades, this is considered a performance success.

While there are predictors such as prior GPA, and test scores that have been used to predict performance success, it is important to explore other factors that can be used to reevaluate current student success prediction. There is an increasing research interest in educational data mining (EDM) (Romero and Ventura 137). EDM is a growing field that uses machine learning techniques for insights from educational data. For example, the insights can help the educators to improve teaching techniques, to understand learners, to improve learning process and it could be used by a learner to improve their learning activities. This could also help administrators in making the right decisions when allocating resources that yield optimal outcomes to make sure students are getting the most out of the education.

1.1 Objectives

The goal of the project is to create a web application that can provide students and teachers with performance prediction based on different student attributes. Obtaining the performance metrics through the application should be seamless for users. A user selects different characteristics of a student on the application and once the user hits the submit button, the student is classified in a low, middle or high performance class and displayed on the screen. On the client side, predictions would be presented by visualizations that highlight the performance class. On the server side, Python will be responsible in executing the predictive results on the web application. To achieve this, implementation of the web application is not enough. Various machine learning techniques will be implemented and we will derive the optimal predictive model to be used in the web application.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 2

Background

This section aims at giving an overview of the students' performance dataset. We examine the dataset that is used in this project, and give explanation of the features that are used in building the prediction model.

2.1 Schools and Student Performance

As more people choose and can pursue an education, so is the need to make sure that students succeed and learn. The National Center for Education statistics works with different educational institution to study ways to identify students that are not performing well. These findings can lead to much needed help to educational institutions in directing support resources where they are needed most(NCES).

However, with all these measures taken to ensure that students are supported as much as possible, resources can be overwhelmed by lack of better tracking and identifying students that are most in need of help. As intervention programs continue to grow across different educational institutions to close achievement gaps, there are still huge gaps in the technological systems that support and enhance the intervention programs (Shahiri and Husain 416).

Besides overall trends on student success, a student performance can vary and greatly be influenced by different factors. As educational institutions implement and augment intervention systems, given the different attributes that a student profile has, this project aims at modeling the relationship between student performance and the various factors that affect it by classifying performance as low-level, middle-level and high-level.

2.1 Dataset

The dataset was obtained from Kaggle. The dataset is comma delimitated file called Students' Academic Performance Dataset (xAPI-Edu-Data). It is a multivariate dataset with four hundred and eighty students and sixteen attributes. The features are classified into three major categories: Demographic features such as gender and nationality. Academic background features such as educational stage, grade Level and section. Behavioral features such as raised hand in class, opening resources, answering survey by parents, and school satisfaction (Kaggle)

Ibrahim Aljara, the individual who posted the dataset on Kaggle gives a background of the student's academic performance dataset. The data is collected using a learner activity tracker tool, which is called EXPERIENCE API (xAPI). The xAPI is a component of the training and learning architecture (TLA) that enables to monitor learning progress and learner's actions like reading an article or watching a training video.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

List of Predictor Features:

Gender	Nominal	Male or Female
Nationality	Nominal	Kuwait, Lebanon, Egypt, Saudi Arabia, USA, Jordan, Venezuela, Iran, Tunis, Morocco, Syria, Palestine, Iraq, Libya
Place of Birth	Nominal	Kuwait, Lebanon, Egypt, Saudi Arabia, USA, Jordan, Venezuela, Iran, Tunis, Morocco, Syria, Palestine, Iraq, Libya
Educational Stages	Nominal	lower level, Middle School, High School’
Grade Levels	Nominal	G-01, G-02, G-03, G-04, G-05, G-06, G-07, G-08, G-09, G-10, G-11, G-12
Section ID	Nominal	A, B, C
Topic	Nominal	English, Spanish, French, Arabic, IT, Math, Chemistry, Biology, Science, History, Quran, Geology
Semester	Nominal	First, Second
Parent responsible for student	Nominal	Mom, Father
Raised hand	Numeric	0-100
Visited resources	Numeric	0-100
Viewing announcements	Numeric	0-100
Discussion groups	Numeric	0-100
Parent Answering Survey	Nominal	Yes, No
Parent School Satisfaction	Nominal	Yes, No
Student Absence Days	Nominal	above-7, under-7

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

The target features:

The students are classified into three numerical intervals based on their total grade or mark as shown in the table below. In this project, these target features will be the one we will be predicting.

Performance	Nominal	L, M, H

L	Low-Level	0-69
M	Middle-Level	70-89
H	High-Level	90-100

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 3

Technologies and Algorithms Used

For this thesis, it is essential to use appropriate machine learning tools that will facilitate the project purpose. The different tools mentioned below encompass the capabilities of storing, analyzing, modeling and building of a predictive web application. All the tools mentioned below are open sourced tools that can be downloaded on the internet and require minimal amount in setting up their environments.

3.1 Python

A high-level programming language, Python is used in this research for analysis, modeling and web development. Python having clear syntax, with preprogrammed high level data types makes it easy to program. Python has many tutorials and references on the internet, which make learning and implementation of projects in Python fast. Because of its popularity, this means there are many modules available for many applications. Python is popular in the scientific community, and this makes it the ideal candidate to do mathematical computations much faster because of the different scientific packages available to it (Python)

3.2 Jupyter Notebook

The Jupyter Notebook is an interactive computing environment that enables users to author notebook documents that include: live code, interactive widgets, plots, narrative text, equations, images and video. The Jupyter Notebook has three different components: The Notebook Web Application, Kernels and Notebook Documents.

An interactive web application for writing and running code interactively and authoring notebook documents, the Notebook Web Application is essential to exploring the data and building predictive models. Kernels separate processes started by the notebook web application that runs user's code in each language and returns output back to the notebook web application. kernels also handle things like computations for interactive widgets, tab completion and introspection. Lastly Notebook Documents are self-contained documents that contain a representation of all the content visible in the notebook web application, including inputs and outputs of the computations, narrative text, equations, images, and rich media representations of objects. (Project Jupyter)

3.3 Flask- A Python Micro framework

To develop the web app for this thesis, Flask, a Python micro web framework, is utilized. Flask as a micro framework means that, a developer can extend to other frameworks of their choice to develop an application for example hypertext markup language, cascading styles, JavaScript and more. Flask does not include a database abstraction layer, form validation or anything else where different libraries already exist that can handle that. Flask supports extensions to add to such functionality to a web application (Flask)

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

3.4 Machine Learning Packages

Three packages are used to carry out different plotting, modeling, and analysis techniques in this thesis. Numpy (Numerical Python) provides an efficient interface to store and operate on large amounts of data. NumPy arrays are useful because they provide more efficient storage and data operations as the arrays grow larger in size. Pandas is a package built on top of NumPy, and provides an efficient implementation of a data frame. Data frames are essentially multidimensional arrays with attached row and column labels, and often with heterogeneous type and/or missing data. Finally, Scikit-learn is a machine learning library in python that features different algorithms and is designed to interoperate with Numpy and pandas. This library will be used to build the predictive models that will be in the flask application for this project. (Combs Chapter 2)

3.5 Categories of Machine Learning

In the field of machine learning, there are three types of machine learning algorithm. Supervised learning, unsupervised learning and reinforcement learning algorithms. Supervised learning algorithms learn to make predictions based on a set of features. An example of supervised learning is the ability to classify emails that are spam or not. From a dataset of emails that are classified as spam or not, the algorithm learns a pattern and when it is exposed to new emails, the algorithm is able to correctly classify the new emails. In unsupervised learning, algorithms are dealing with unlabeled features. Here the algorithm tries to learn some pattern in the dataset without prior training. An example of unsupervised learning can be for an algorithm to put into clusters different customer segments based on different characteristics. The work of unsupervised machine learning algorithms come in handy when organizing or trying to find some structure to describe the dataset. Reinforcement learning algorithms choose actions based on response to each data point. An example of reinforcement learning is an algorithm trying to learn how to play a chess game. To perform reinforcement learning, you define a reward system such as the algorithm is doing well when it wins and continues to learn when it loses (Gareth, Witten, Hastie and Tibshirani 45)

For this project, supervised learning algorithms will be implemented for prediction. Though there are pros and cons to using a supervised learning approach. The advantages are that we can train the algorithm on a subset of the dataset and able to test the algorithm on new features. We can do this process multiple times for accuracy predictions. The disadvantages are that we must carefully chose and test different algorithms for the problem we are trying to solve, and this can take a lot of time in training and testing the algorithm.

Within the family of supervised machine learning algorithms, there are two types; regression and classification. Classification algorithms will be used in this project because we are trying to predict to which of a set of categories (low, medium or high) a new observation(student) belongs to. But first we need to understand the difference between regression and classification in supervised learning methods.

Regression, a supervised method that models a feature y based on a feature x is a popular method in modeling. An example of regression is trying to predict the housing price of a home in an area based on the number of rooms. This will be done by taking different housing prices and number of rooms of those houses and fitting a line. The example below shows how a regression

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

is performed. We have predictor feature x and a target feature y, we fit a straight line to this data that minimizes the average squared distance between the data points and the fitted line.

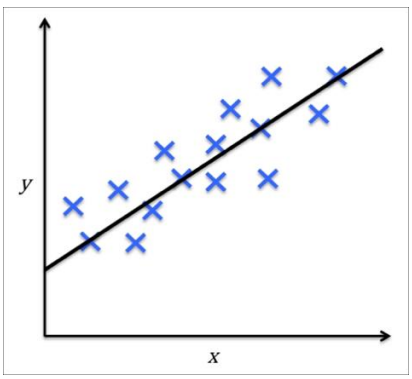

Figure 3.5.1 Linear Regression Example

Classification, the method we will be using for this project is a process of algorithms to predict between two or more categories. When predicting between two features, this is called binomial classification, when the algorithm is predicting between multiple features, it is called a multiclass classification. For this project, the problem being addressed is a multiclass classification. The aim is to predict whether a student performance is low, medium or high based on different student characteristics. As shown in the figure below, to better understand classification we have predictor features x_1 and x_2 and we use them to classify where the new data points belong between the two features.

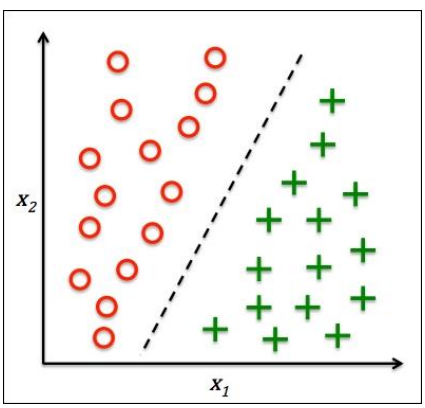

Figure 3.5.2: Classification Example

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 4

A Framework for Machine Learning in Predictive Modeling

This section outlines a conceptual framework for the process that is used in building a predictive model for this project, from data exploration and preprocessing to selecting the best model. While carrying out this project, the general framework used to build the model follows these steps: Explore the dataset, preprocess the data and then build predictive models (train and test models)

4.1 Exploring and preprocessing data

Exploration of data is crucial to understanding the dataset and identifying ways the data will be manipulated. Data exploration involves making informative visualizations that highlight relationships between features. Another important aspect of data exploration is to understand what features are going to be transformed. In case of categorical features, they will be transformed into numerical features to perform predictive analytics.

Preprocessing data is important to the practice of machine learning because, in the real world, data is sometimes incomplete and or inconsistent. Most of the times data stored in files or databases is not in the right format to build predictive models. Pandas, a Python package for manipulating data frames along with the built-in Python language features provides high-level, flexible, and fast set of tools to manipulate data into the right form when faced with missing data, duplicate data and some other analytical data transformations. Manipulating or preprocessing data also called feature engineering is therefore essential into turning data into features that we can use to build predictive models.

4.2 Building models (train and test)

One important practice that distinguishes good modeling from bad modeling is how a model is validated. Model validation is estimating how effective a model is by applying to it some of the training data and comparing the prediction to the known value. To get a better sense of the model performance, this can be done by a technique called cross validation. At the core, performing cross validation is, to hold back some subset of the data from the training of the model and then use that set to check or validate the model performance. In this thesis, the way cross validation is done, is by doing a sequence of fits where each subset of the data is used as both a training set and as a validation set (Vanderplas 67).

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

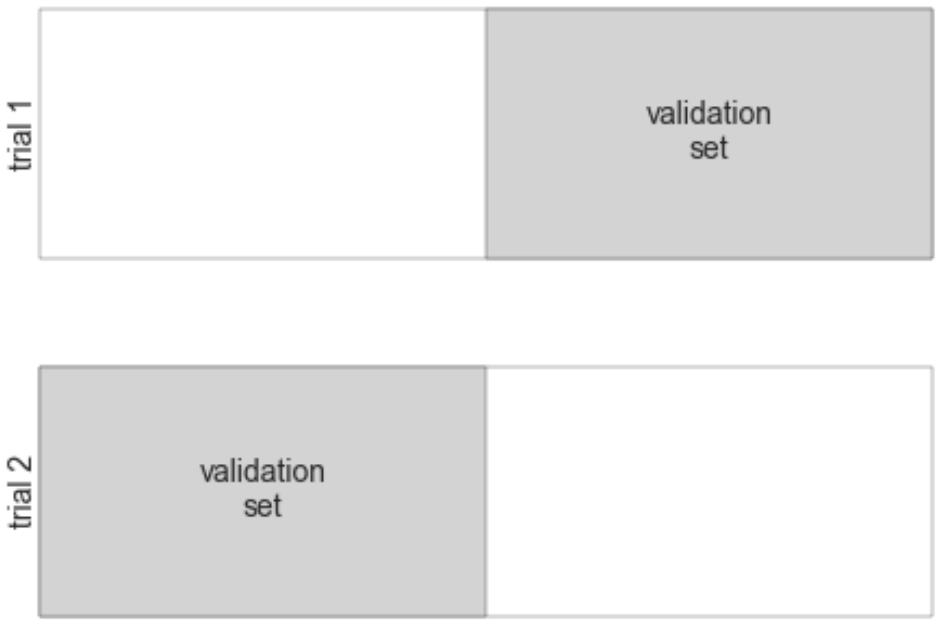

Figure 4.3.1: An example of a two-fold cross-validation

Figure 4.3.1 above shows an example of how cross-validation is done. We use each subset as a training set. This process can be expanded further to more folds. Validating a model across different subsets can better inform us how the algorithm is performing and accurately predicting values across different subset of the dataset.

After validating the model, model selection play a crucial role in the practice of machine learning. Of core importance is the following question: if the model is bad at predicting new features, how can we proceed? To solve such a question on finding the best model one must consider a trade-off between bias and variance.

Bias is when a model under fits the data, this can be seen in the figure below on the left as the straight-line attempts to fit the data but it does not do well at describing the dataset. This means that this model is not flexible enough to account all the features in the data. And as we test the model on new data, the model is not able to predict well.

Variance as seen in the figure below on the right is when a model attempts to fit all the data, accounting all the features in the data. This type of model has a lot of flexibility that the model ends up accounting for random errors as well as underling data distributions. This is not ideal for modeling, because when we test the model on new data, it does worse at predicting new features.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Figure 4.3.2: Bias vs Variance

After completing the tasks above, one must have metrics to compare the different models to evaluate their errors and be able to improve prediction performance. The process above is a framework that is used to carry out predictive modeling for different problems. It is a good first step in understanding a problem and allows us to explore more complex analysis. The section below will explore in more detail the classification methods that are used in this project.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 5

Classification Methods

In this section, I discuss why classification algorithms are necessary to carry out the prediction of student's performance. This section goes further in explaining the fundamentals of the classification methods used for the duration of the thesis project.

The problem of predicting students' performance can be considered a classification problem, since we are concerned at predicting values that can fall within a discrete range of outputs. Through different classification methods, we will be able to explore further the relationship between the predictor features in the dataset and the target feature we will predict. The process of selecting the best classification algorithm for the thesis problem requires training and testing different algorithms. Each algorithm has its own strengths based on assumptions. It is going to be important in terms of evaluating the results to compare a few algorithms to come up with the best performing algorithm.

For this project, we will look closely at these algorithms: Multinomial logistic regression, support vector machines, and random forest. It is important to note that, the performance of the predictive model usually comes down to the underlying data that is used for modeling.

5.1 Logistic regression

A powerful method for modeling class probabilities, logistic regression is a popular model that is used in a lot of industries such as healthcare where, it can be used to analyze the likelihood of a patient to be readmitted within thirty days of discharge(Yerex). Logistic regression is an algorithm that aims at modeling the relationship between a binary class and other classes that are assumed to be related. Logistic regression being a model for binary classes can also be extended to multiclass classification (Raschka 23). The figure below highlights how a logistic regression works as a sigmoidal curve:

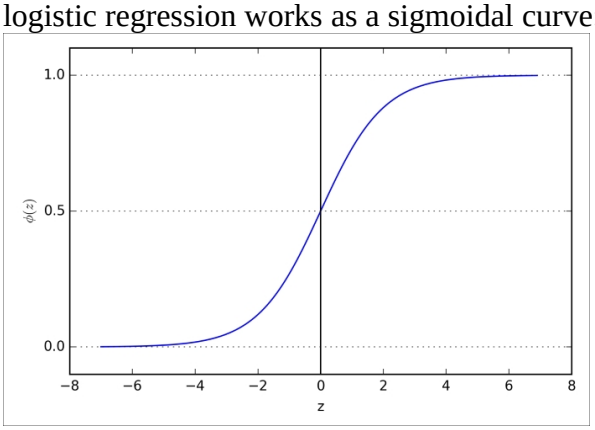

Figure 5.2.1: Logistic Regression Example

Logistic regression as a probabilistic model can be explained by the odds ratio. The odds ratio is written as $\frac{p}{(1-p)}$, where p is the probability of the positive event. To further define the logit function, we can simply call it the logarithm of the odds ratio.

$$logit(p) = log\frac{p}{(1-p)}$$

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

This logit function takes impute values in the range of 0 to 1 and transforms them to values over the entire real number range, which we can use to express a linear relationship between feature values and the log odds

$$logit\left(p\left(y=1\mid \boldsymbol{x}\right)\right)=w_0x_0+w_1x_1+\cdots+w_mx_m=\sum_{i=0}^{m}w_ix_i=\boldsymbol{w}^T\boldsymbol{x}$$

Where, p(y=1|x)) is the conditional probability that a sample belongs to class 1 given features x. To predict the probability that a certain sample belongs to a class, we then take the inverse form of the logit function. Which is the logistic function.

$$\phi(z)=\frac{1}{1+e^{-z}}$$

The above function is the foundation of building a logistic regression model that is already implemented in Scikit-learn a Python library for machine learning algorithms. And we can use Scikit-learn, to fit the data and find probabilities that a feature belongs to certain class, whether it's the low, middle or high student class in student's performance.

5. 2 Support Vector Machines

Support Vector Machine (SVM) is a powerful algorithm that is used by many people in machine learning field. The goal of the SVM optimization is to maximize the margin, which is the distance between the separating hyperplane and the training data that are closest to this hyperplane. The data that are closest to the hyperplane, are called support- vectors (Raschka 26) The figure below highlights the building blocks of a SVM:

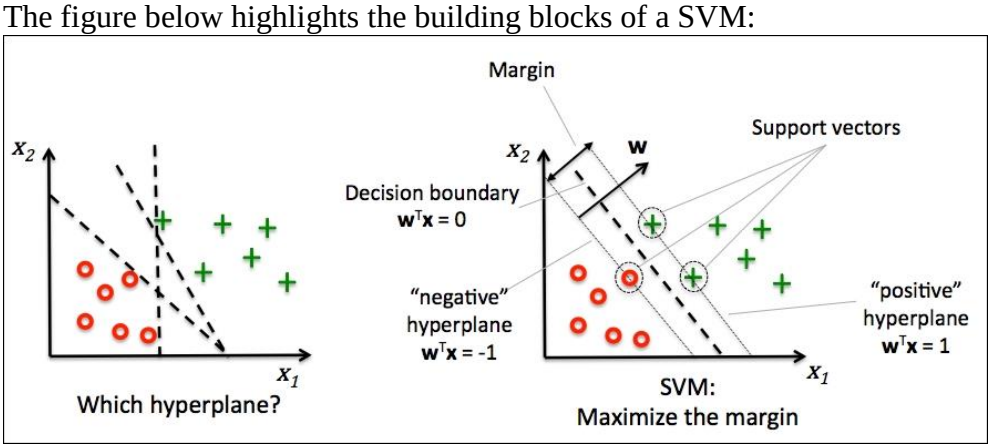

Figure 5.3.1: Support Vector Machine Example

From figure 5.3.1 above, it is important to understand the rationale behind maximizing margins and how to best do it. On the left image, we have samples on the graph that we want to classify. To classify the points, we must maximize the distance or margin between the features and the hyperplanes. A hyperplane is a decision boundary, and large margins tend to have lower generalization error while small margins can cause problems such as overfitting that was explained in the section earlier on bias and variance. The goal of the SVM can further be explained by the equations below.

Take a positive and a negative hyperplane:

$$w_0+\boldsymbol{w}^T\boldsymbol{x}_{pos}=1 \qquad (1)$$

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

$$w_0 + \boldsymbol{w}^T \boldsymbol{x}_{neg} = -1 \quad (2)$$

Subtract the equations above and you get:

$$\Rightarrow \boldsymbol{w}^T \left(\boldsymbol{x}_{pos} - \boldsymbol{x}_{neg} \right) = 2$$

Normalize this equation by the length of the vector w, which is this:

$$\|\boldsymbol{w}\| = \sqrt{\sum_{j=1}^{m} w_j^2}$$

Finally, we have this equation:

$$\frac{\boldsymbol{w}^T \left(\boldsymbol{x}_{pos} - \boldsymbol{x}_{neg} \right)}{\|\boldsymbol{w}\|} = \frac{2}{\|\boldsymbol{w}\|}$$

The left side of the equation is the distance between the negative and positive hyperplanes and that is what we want to maximize. Now the objective functions of the SVM is to maximize the margin by maximizing

$\frac{2}{\|\boldsymbol{w}\|}$ under the constraints that the samples are classified correctly, which is shown in the equation below

$$w_0 + \boldsymbol{w}^T \boldsymbol{x}^{(i)} \geq 1 \; if \; y^{(i)} = 1$$

$$w_0 + \boldsymbol{w}^T \boldsymbol{x}^{(i)} < -1 \; if \; y^{(i)} = -1$$

To interpret the equations above, we can say that all negative samples should fall on one side of the negative hyperplane and positive samples should fall behind the positive hyperplane. Though they are detailed math behind this algorithm. Using Scikit-learn, we will be able to use this algorithm seamlessly.

5.3 Random Forest

To understand the random forest algorithm, it is important to look at decision tree learning algorithm first. A decision tree in its simplest form is a model that breaks down the data by making decisions based on prompting a series of questions. The example in the figure below showcases a decision tree:

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

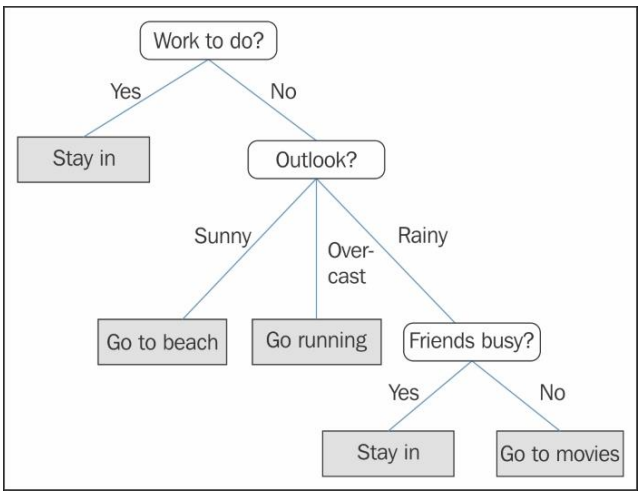

Figure 5.4.1: Decision Tree Example

Based on the variable or features of the training set, the decision tree algorithm learns a series of questions to infer to class labels of the samples. This algorithm commences at the tree root and split data on the feature that results in the largest information gain(IG). This process can be performed repeatedly.

We apply this method until all samples at each node belong to the same class (Raschka 34). However, while performing this method, it is important to keep in mind the deeper the tree, the more we are prone to overfitting. So, we control for this, by setting a maximal depth of tree. As shown in figure 5.4.1 the depth of the tree is three. So, if we were to set the depth of the tree to thirty, we are prone to overfitting.

For fundamentals on creating a tree, lets define a function that will be optimized for decision tree learning. The objective function is to maximize IG at each split:

$$IG\left(D_p, f\right) = I\left(D_p\right) - \sum_{j=1}^{m} \frac{N_j}{N_p} I\left(D_j\right)$$

Here, f is a feature to perform the split, D_p and D_j are the dataset of the parent and j^{th} child node, I is the impurity measure, N^P is the total number of samples at the parent node, and N, is the number of samples in the jth child node. The information gain is simply the difference between the impurity of the parent node and the sum of the child node impurities, the lower the impurities of the child nodes, the larger the information gain.

Now that a decision tree has been explained, an algorithm that is used for the students' performance classifier is random forests. Random forests are considered an ensemble of decision trees. The fundamentals behind this method is that, combining week learners can result in a strong learner to build a model.

The random forest algorithm works by first, drawing a random bootstrap sample of size n and then grow a tree from the bootstrap sample, at each node. Perform 1 to 2^k times of random bootstrap sampling, after aggregate the prediction of each tree to assign the class label by majority vote or as it is called ensemble learning.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

5.4 Summary

Looking back at the classification models mentioned above, we are equipped with processes and knowledge to implement these models to predicting students' performance. In the analysis section, we will showcase how all these algorithms perform and how to evaluate and compare the predictive power of these algorithms.

Section 6

Analysis

In this section, I will lay the background of the project in predicting student performance by using all the technologies and process mentioned above. This in-depth analysis will cover the pipeline of creating the classification model by explaining the background of the problem, the data, methods and algorithms used, the different results of the algorithms, the best performing algorithm, and model interpretation

6.1 Data Visualization and Preprocessing

The visualizations below allow us to inspect the data and using different plotting methods we can pull insights from the dataset. The dataset is dissected to learn how different features are connected and what is going to be important in terms of predicting the target feature. Since the dataset consists of four numerical features, we can examine them and how they relate to the target feature.

First, figure 6.1.1 is an ensemble of plots that show the relationship between, raising hands, visiting resources, viewing announcements and attending discussions as they relate to a student being in a low, middle or high student subclass. In detail, we can examine how different features relate to each other.

Starting with the relationship between raising hands to discussion on whether a student will be classified in a low, middle or high student class. We observe that raising hands does not necessary show that a student will attend discussions or raising hands and attending discussions does not guarantee a student to be a high performing student. When we observe the relationship between raising hands and announcement view, there is a trend that, the more you raise hands in class, you tend to look at announcements hence affecting the performance in school. The relationship between visiting resources and raising hands show that many of the high performing students had raised hands and visited resources more than students performing at the middle or low level. Therefore, raising hands can be a factor in student performance but is not enough of an observation to predict student performance.

In terms of visiting resources, from the figure below, we can observe that visiting resources on campus can lead to a student to fall in the low, middle and high performance class. However, students who visited resources did not attend discussions. Visiting resources and looking at announcements features are best are informing us as shown in the figure 6.1.1 the performance group a student is likely to be part of.

From the figure 6.1.1 below, trends in how raising hands in class, visiting resources, looking at announcements and attending discussion play a role in performing at a low, middle or high level, can inform us about the model performance when it comes to evaluating a model and understanding features that are crucial in predicting student performance. The analysis below

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

highlights the counts of observations between the numerical features and target features. Overall, we can see that visiting resources and raising hands in class have a much more impact on performance than looking at announcements and attending discussions.

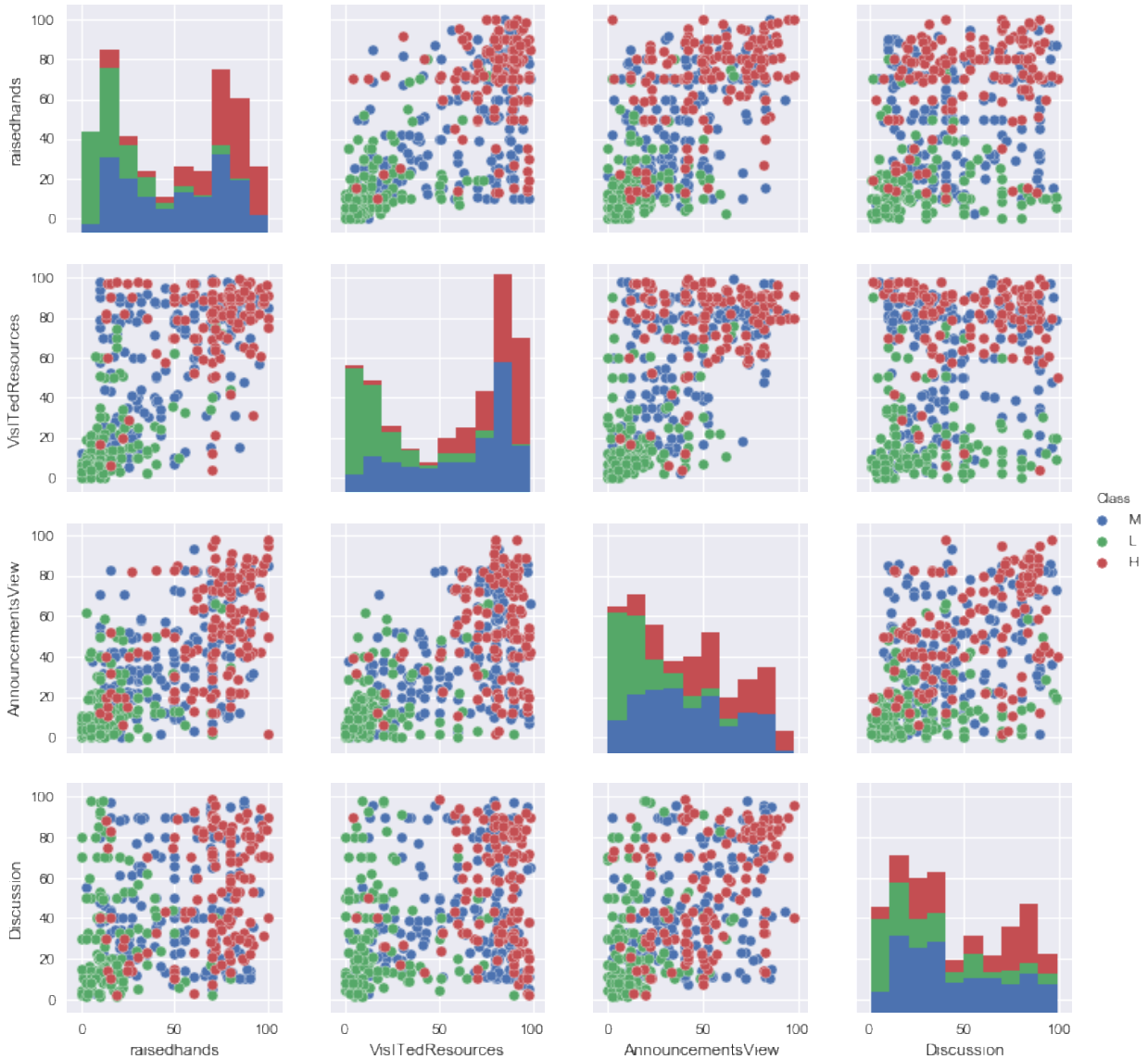

Figure 6.1.1: This plot shows the relationship between the numerical predictor features in the dataset and the target features

The analysis of the four numerical features is examined further by the correlation map or correlation analysis as shown in figure 6.1.2. The map shows the dependence between the features, and it highlights the correlation coefficients between the predictor features and target features. The value one highlights that the features are very strongly correlated and has a positive linear relationship, and as values become less than one, the positive linear relationship between features changes. At zero, there is no linear relationship. At a value of negative one there is a perfect negative linear relationship. This analysis further emphasizes the observations between

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

features in figure 6.1.1 and can inform us on predictor features that are the most important in predicting student performance.

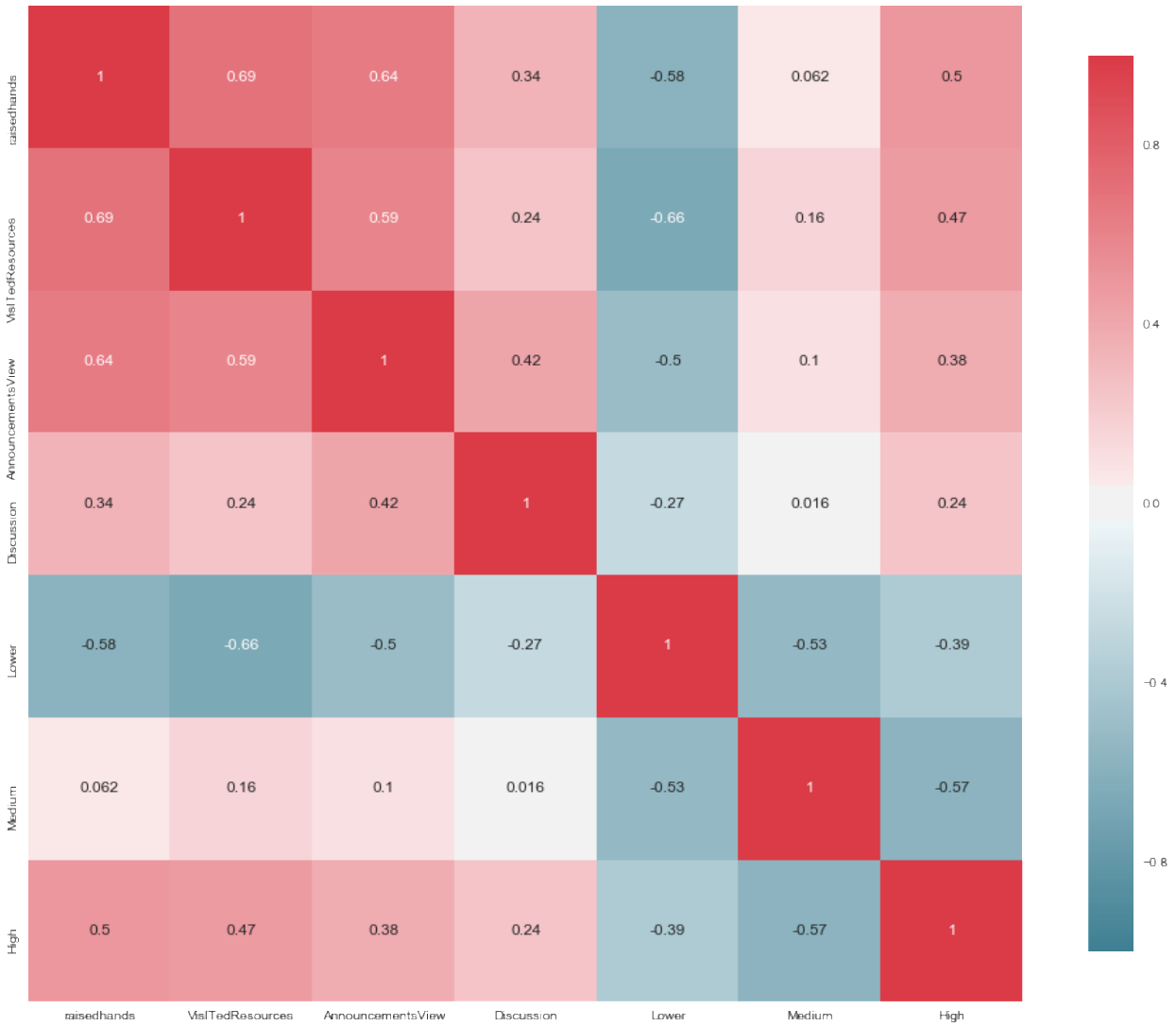

Figure 6.1.2: Correlation Map

In figure 6.1.3, we examine the categorical variables with count plots. The aim is to visualize the observations in each categorical bin using bars. From the figure below, we can see the difference in gender across the dataset, with the school having more males than females. The most represented nationalities, grades with the most students, students' relation to their parents and the most popular topics that students take. This analysis can further highlight how data points are distributed across the dataset.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

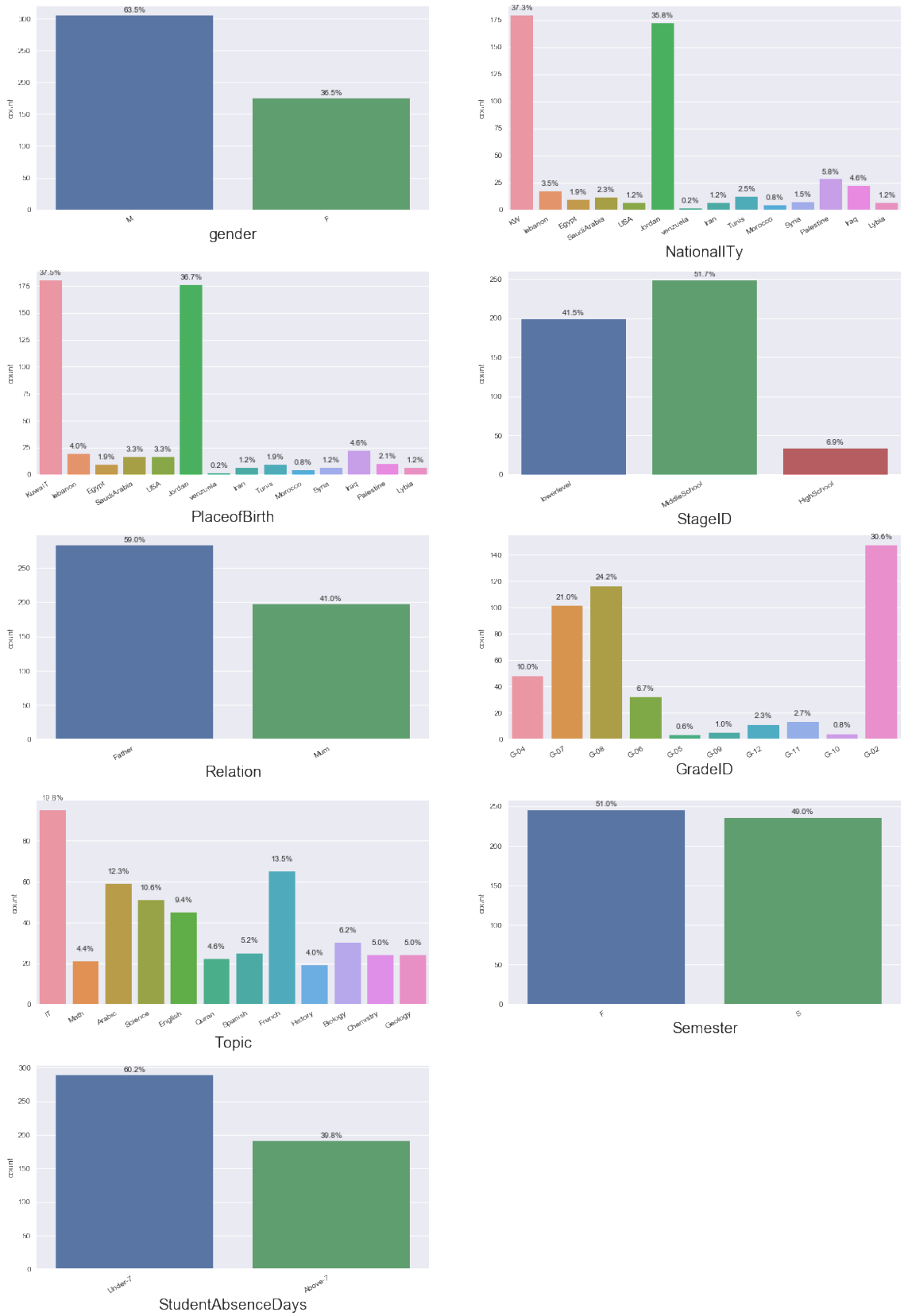

Figure 6.1.3: Categorical Features visualization

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

After exploring different relationships between the features and having a clear sense of the dataset. We can proceed to building the predictive models and comparing how different algorithms perform on the dataset. The next section will highlight the model results, interpretation and the features that are most important in predicting student performance.

6.2 Model Results and Interpretation

For this project, the log loss, also known as logistic loss is going to be the metric in terms of evaluating the best model to use in the web application. To calculate the log loss, the algorithm must assign a probability to each class. The log loss can be understood as the probabilistic confidence of the model to assign a feature in the right class.

This loss function is used in (multinomial) logistic regression and is defined as the negative log-likelihood of the true labels given a probabilistic classifier's predictions. The log loss is only defined for two or more labels and can be easly calculated using the scikit learn library in python. (Scikit-learn.org)

To further understand the logarithmic loss, for a single sample with true label y_t in {0,1} and estimated probability y_p that y_t = 1, the log loss is:
-log P (y_t |y_p) = -(y_t log(y_p) + (1 - y_t) log (1 - y_p)):

To better understand how the algorithms are performing, the confusion matrices in figures 6.2.1, 6.2.2, 6.2.3 and 6.2.4 for the different models are shown. A confusion matrix is a table that is used to describe the performance of a classification algorithm on the test data. The table shows true label vs predicted labels for the models created.

To determine the competitiveness and accuracy of the models above, we compare the log-loss estimates of the four predictive models. As seen in table 6.3.6, The random forest model has the lowest log loss compared to the rest of the models.

Algorithm	Log-loss
Logistic Regression	0.65
Support Vector Machines	0.60
Decision Tree	1.53
Random Forest	0.51

Table 6.2.6: Log-loss results of the prediction models

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

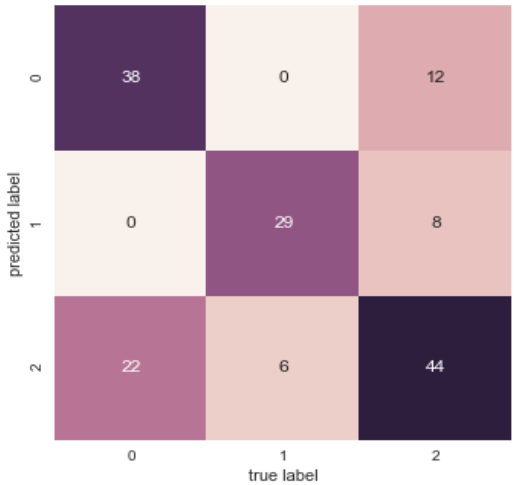

Figure 6.2.1: Confusion Matrix Logistic Regression

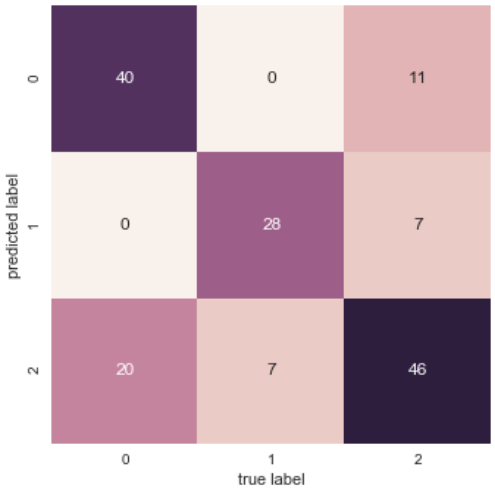

Figure 6.2.2: Confusion Matrix Support Vector Machine

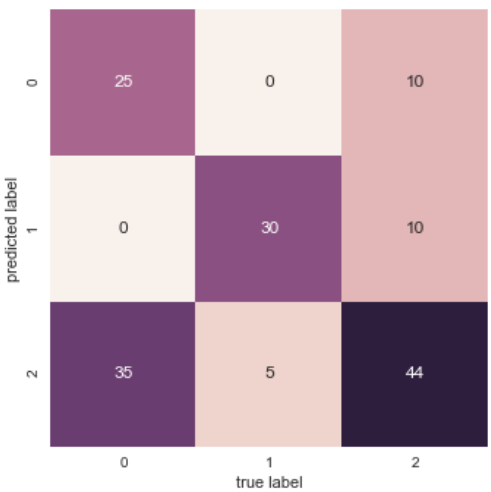

Figure 6.2.3: Confusion Matrix Decision Tree

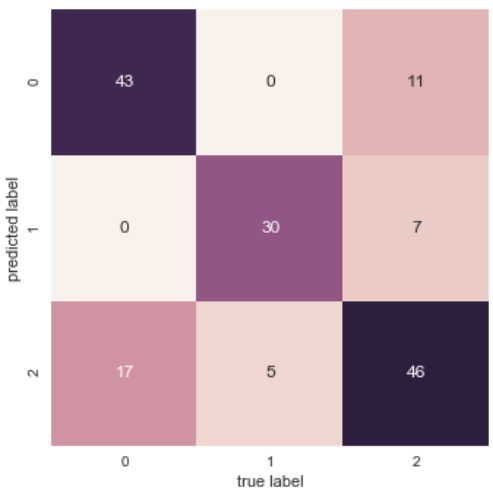

Figure 6.2.4: Confusion Matrix Random

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Only one trial is not enough to be confident of the models we have built. Different tests are done to evaluate how different models behave when certain features are removed or added. Instead of using all the initial fifteen features in the dataset, identifying features that are the most important in predicting student performance and running the models again and analyzing how they perform will further inform the features that are strong predictors of student performance.

To check for feature importance, we use a Scikit-learn method to check for feature importance in the dataset. This is shown in figure below.

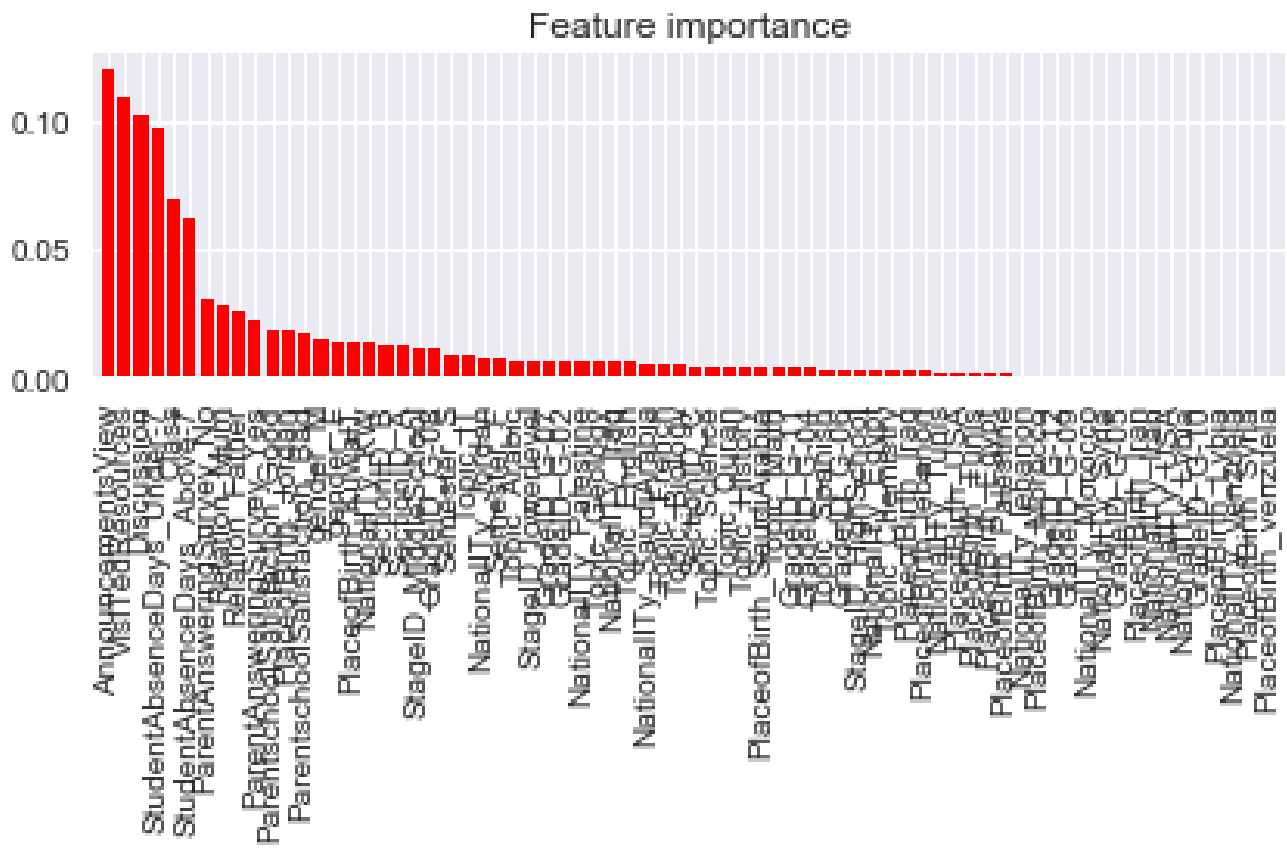

Figure 6.2.7 Important features in predicting student performance

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

To run a second trial of models, we select features that are above 0.05 in figure 6.2.7, these features are announcements view, visited resources, discussion, student absence days. We add five more features that show significance which are parent answering survey, relation to the parent, place of birth, parent school satisfaction and gender. This process can be done as many times as possible to test how different features inform the model in predicting student performance by removing non-significant features and testing how the models perform.

The table below highlight the new log loss results of the nine features.

Algorithm	Original Model Log-Loss	New Model Log-loss
Logistic Regression	0.65	0.65
Support Vector Machines	0.60	0.59
Random Forest	0.51	0.49

Table 6.2.8: New Results

Table 6.2.8 shows that after omitting almost half of the features, the log loss of logistic regression did not change, the log loss of the support vector machines did improve by a little bit and so is the random forest. We can conclude that, the random forest algorithm does a better job at modeling student performance than the rest of the algorithms. Therefore, we will use the random forest model to build a predictive web app that can be useful for users to make sense of the model.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 7

Web Application

This section covers the screen shots of the real application. In the web application, the window in figure 7.1 shows the input selection of the different features that have been discussed in this project. The user needs to select the inputs below to make predictions.

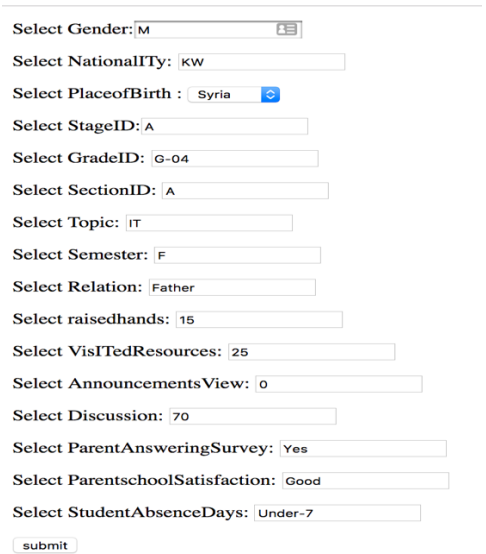

Figure 7.1: Inputs Selection Window

After selecting the appropriate inputs and clicking the submit button, prediction results are displayed in a new window. Below is the screen that showcase the prediction results as a label class as show in figure 7.2. Because the feature target “Class” is coded as [0,1,2] for Low, Middle, and High, after completing the input selection and clicking the submit button that calls the random forest model function, we get a result of [1]. This result means the student belongs to a middle performance class. This process can be repeated multiple times to derive insights from the model as a web application to classify students.

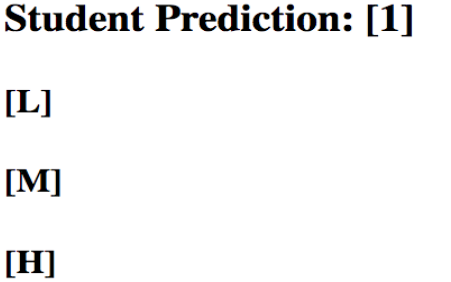

Figure 7.2: Prediction result

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Section 8

Summary and Future Enhancements

We have managed to build a web application that allows users to look at future students' performance. Three classification models have been explored and compared, before arriving at our best model which is a random forest classifier. The success of this approach to creating the web application for generating predictions can be applied to other problem sets concerned with students and academics.

Despite having applied machine learning to predict students' performance and build a web application that met the initial requirements, there are improvements that can be made in the future. A larger dataset on students can be useful in producing robust models. The dataset used is small with only three hundred and fourteen students, the more data we can gather about students, the better the model can learn to classify the students. More data points to train and test on, would improve the results and confidence in the model prediction accuracy. The web application can be further developed for interactivity and better visualizations to better interpret student performance predictions.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

References

A. M. Shahiri and W. Husain, “A Review on Predicting Student's Performance Using Data Mining Techniques”, Proceeding Computer Science, vol. 72, (2015), pp. 414-422.
National Center for Education Statistics (NCES) Home Page, a Part of the U.S. Department of Education. N.p., n.d. Web. 17 Apr. 2017.

C. Romero and S. Ventura, “Educational data mining: A survey from 1995 to 2005”, Expert systems with applications, vol. 33, no. 1, (2007), pp. 135-146.

Combs, Alexander T. "Chapter 2." *Python Machine Learning Blueprints: Intuitive Data Projects You Can Relate To: An Approachable Guide to Applying Advanced Machine Learning Methods to Everyday Problems*. Birmingham, UK: Packt, 2016. N. pag. Print.

Flask (A Python Microframework). N.p., n.d. Web. 17 Apr. 2017.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: With Applications in R*. N.p.: n.p., n.d. Print.

Idris, Ivan. *Python Data Analysis*. N.p.: n.p., 2014. Print.

Kaggle: Your Home for Data Science. N.p., n.d. Web. 17 Apr. 2017.

Python.org. N.p., n.d. Web. 17 Apr. 2017.

Project Jupyter. N.p., n.d. Web. 17 Apr. 2017.

Raschka, Sebastian, and Randal S. Olson. "Chapter 3." *Python Machine Learning: Unlock Deeper Insights into Machine Learning with This Vital Guide to Cutting-edge Predictive Analytics*. Birmingham: Packt, 2016. N. pag. Print.
"3.3. Model Evaluation: Quantifying the Quality of Predictions¶." *3.3. Model Evaluation: Quantifying the Quality of Predictions — Scikit-learn 0.18.1 Documentation*. N.p., n.d. Web. 17 Apr. 2017

Vanderplas, Jacob T. "Chapter 5." *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O'Reilly Media, 2017. 2-5. Print.

Yerex, Robert P. *A Predictive Model of Patient Readmission Using Combined ICD-9 Codes as Engineered Features* (n.d.): n. pag. Web.

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Appendix A: Code Snippet-Visualization

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

data = pd.read_csv("~/Desktop/xAPI-Edu-Data.csv")
data.head(10)

# check original shape
print(data.shape)

data.info()

# view columns using df.columns
data.columns

data.isnull().sum()

#breakdown by class
sns.countplot(x="Topic", data=data, palette="muted");
plt.show()

data['Lower'] = np.where(data['Class']=='L',1,0)
sns.factorplot('Topic','Lower',data=data,size=9)

data['Medium'] = np.where(data['Class']=='M',1,0)
sns.factorplot('Topic','Medium',data=data,size=9)


data['High'] = np.where(data['Class']=='H',1,0)
sns.factorplot('Topic','High',data=data,size=9)

sns.countplot(x='Class',data=data,palette='PuBu')
plt.show()

data.Class.value_counts()

sns.countplot(x='ParentschoolSatisfaction',data = data, hue='Class',palette='bright')
plt.show()

sns.countplot(x='gender',data = data, hue='Class',palette='bright')
plt.show()

```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
sns.countplot(x='StageID',data = data, hue='Class',palette='bright')
plt.show()

sns.countplot(x='GradeID',data = data, hue='Class',palette='bright')
plt.show()

def plot_correlation_map( data ):
    corr = data.corr()
    _ , ax = plt.subplots( figsize =( 18 , 14 ) )
    cmap = sns.diverging_palette( 220 , 10 , as_cmap = True )
    _ = sns.heatmap(
        corr,
        cmap = cmap,
        square=True,
        cbar_kws={ 'shrink' : .9 },
        ax=ax,
        annot = True,
        annot_kws = { 'fontsize' : 12 }
    )

plot_correlation_map(data)

cat_Vars = ['gender', 'NationalITy', 'PlaceofBirth', 'StageID','Relation', 'GradeID', 'Topic', 'Semester', 'StudentAbsenceDays']

fig = plt.figure(figsize=(20, 30))
fig.subplots_adjust(hspace=.3, wspace=0.2)

for i in range(1,len(cat_Vars)+1,1):
    ax = fig.add_subplot(5, 2, i,)
    sns.countplot(data[cat_Vars[i-1]])
    ax.xaxis.label.set_size(20)
    plt.setp(ax.get_xticklabels(), rotation=30, horizontalalignment='right')
    total = float(len(data))
    for p in ax.patches:
        height = p.get_height()
        ax.text(p.get_x()+p.get_width()/2.,height + 5,'{:1.1f}%'.format(100 * height/total),ha="center")

        num_Vars = [ 'raisedhands','VisITedResources', 'AnnouncementsView', 'Discussion', 'Class']

        sns.pairplot(data[num_Vars], hue='Class')

data.head()
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

Appendix B: Code snippet-Building models and data preprocessing

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
from sklearn import cross_validation
from sklearn.cross_validation import cross_val_score

data = pd.read_csv("~/Desktop/studentapp/xAPI-Edu-Data.csv")


data.head(10)

for column in ['Class']:
    data[column] = data[column].astype('category')
    data[column] = data[column].cat.codes

data.head()

data.columns

data = pd.get_dummies(data)

data.head()

data.columns

from sklearn.cross_validation import train_test_split

X_train, X_test, y_train, y_test = train_test_split(data.drop('Class',axis=1),data['Class'], test_size=0.33, random_state=1)

from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
model.fit(X_train, y_train)

labels = model.predict(X_test)
labels_1 = model.predict_proba(X_test)

from sklearn.metrics import log_loss
log_loss(y_test, labels_1)
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
42.
43. from sklearn.metrics import accuracy_score
44. accuracy_score(y_test, labels)
45.
46. from sklearn.metrics import confusion_matrix
47. mat = confusion_matrix(y_test, labels)
48. sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False,)
49. plt.xlabel('true label')
50. plt.ylabel('predicted label');
51.
52. from sklearn.linear_model import LogisticRegression
53. model2 = LogisticRegression(C=100.0,random_state=0)
54. model2.fit(X_train,y_train)
55.
56. labels2 = model2.predict(X_test)
57. labels_2 = model2.predict_proba(X_test)
58.
59. log_loss(y_test, labels_2)
60.
61. mat2 = confusion_matrix(y_test, labels2)
62. sns.heatmap(mat2.T, square=True, annot=True, fmt='d', cbar=False,)
63. plt.xlabel('true label')
64. plt.ylabel('predicted label');
65.
66. accuracy_score(y_test, labels2)
67.
68. from sklearn.svm import SVC
69. model3 = SVC(kernel='linear', C=1.0, random_state=0, probability=True)
70. model3.fit(X_train,y_train)
71.
72. labels3 = model3.predict(X_test)
73. labels_3 = model3.predict_proba(X_test)
74.
75. mat3 = confusion_matrix(y_test, labels3)
76. sns.heatmap(mat3.T, square=True, annot=True, fmt='d', cbar=False,)
77. plt.xlabel('true label')
78. plt.ylabel('predicted label');
79.
80. accuracy_score(y_test, labels3)
81. log_loss(y_test, labels_3)
82.
83. model4 = SVC(kernel='rbf', C=1.0, random_state=0, gamma=100.0, probability=True)
84.
85. model4.fit(X_train,y_train)
86.
87. labels4 = model4.predict(X_test)
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
labels_4 = model4.predict_proba(X_test)

mat4 = confusion_matrix(y_test, labels4)
sns.heatmap(mat4.T, square=True, annot=True, fmt='d', cbar=False,)
plt.xlabel('true label')
plt.ylabel('predicted label');


log_loss(y_test,labels_4)

from sklearn.tree import DecisionTreeClassifier
model5 = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=0)

model5.fit(X_train,y_train)
labels5 = model5.predict(X_test)
labels_5 = model5.predict_proba(X_test)
mat5 = confusion_matrix(y_test, labels5)
sns.heatmap(mat5.T, square=True, annot=True, fmt='d', cbar=False,)
plt.xlabel('true label')
plt.ylabel('predicted label');

log_loss(y_test,labels_5)

from sklearn.ensemble import RandomForestClassifier

model6 = RandomForestClassifier(criterion='entropy',
                                n_estimators=29,
                                random_state=0)
model6.fit(X_train,y_train)

labels6 = model6.predict(X_test)

labels_6 = model6.predict_proba(X_test)

mat6 = confusion_matrix(y_test, labels6)
sns.heatmap(mat6.T, square=True, annot=True, fmt='d', cbar=False,)
plt.xlabel('true label')
plt.ylabel('predicted label');

log_loss(y_test,labels_6)

from sklearn.neighbors import KNeighborsClassifier
model7 = KNeighborsClassifier(n_neighbors=5,p=2,
                              metric='minkowski')
model7.fit(X_train,y_train)

```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
labels7 = model7.predict(X_test)

labels_7 = model7.predict_proba(X_test)

mat7 = confusion_matrix(y_test, labels6)
sns.heatmap(mat7.T, square=True, annot=True, fmt='d', cbar=False,)
plt.xlabel('true label')
plt.ylabel('predicted label');
  log_loss(y_test,labels_7)
```

Appendix C: Code snippet-Web app

```
from flask import Flask, request, render_template
import pickle
import numpy as np

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/getperformance',methods=['POST','GET'])
def get_delay():
    if request.method=='POST':
        result=request.form

        #Prepare the feature vector for prediction
        pkl_file = open('cat', 'rb')
        index_dict = pickle.load(pkl_file)
        new_vector = np.zeros(len(index_dict))

        try:
            new_vector[index_dict['gender_'+str(result['gender'])]] = 1
        except:
            pass
        try:
            new_vector[index_dict['NationalITy_'+str(result['NationalITy'])]] = 1
        except:
            pass
        try:
            new_vector[index_dict['PlaceofBirth_'+str(result['PlaceofBirth'])]] = 1
        except:
            pass
        try:
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
34.         new_vector[index_dict['StageID_'+str(result['StageID'])]] = 1
35.     except:
36.         pass
37.     try:
38.         new_vector[index_dict['GradeID_'+str(result['GradeID'])]] = 1
39.     except:
40.         pass
41.     try:
42.         new_vector[index_dict['SectionID_'+str(result['SectionID'])]] = 1
43.     except:
44.         pass
45.     try:
46.         new_vector[index_dict['Topic_'+str(result['Topic'])]] = 1
47.     except:
48.         pass
49.     try:
50.         new_vector[index_dict['Semester_'+str(result['Semester'])]] = 1
51.     except:
52.         pass
53.     try:
54.         new_vector[index_dict['Relation_'+str(result['Relation'])]] = 1
55.     except:
56.         pass
57.     try:
58.         new_vector[index_dict['ParentAnsweringSurvey_'+str(result['ParentAnsweringSurve
    y'])]] = 1
59.     except:
60.         pass
61.     try:
62.         new_vector[index_dict['ParentschoolSatisfaction_'+str(result['ParentschoolSatisfactio
    n'])]] = 1
63.     except:
64.         pass
65.     try:
66.         new_vector[index_dict['StudentAbsenceDays_'+str(result['StudentAbsenceDays'])]] =
     1
67.     except:
68.         pass
69.
70.
71.     pkl_file = open('model5.pkl', 'rb')
72.     model5 = pickle.load(pkl_file)
73.     labels5 = model5.predict(new_vector)
74.     labels_5 = model5.predict_proba(new_vector)
75.
76.
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
77.         return render_template('result.html',labels5=labels5)
78.         return render_template('result.html',labels_5=labels_5)
79.
80.
81.
82. if __name__ == '__main__':
83.     app.run()
84. <!DOCTYPE html>
85. <html>
86. <title>Student-Performance-Prediction App</title>
87. <body>
88.
89. <form action = "/getperformance" method="POST">
90.         <p> Select Gender:<input name="gender" type ="text" value="M" /> </p>
91.         <p> Select NationalITy: <input name="NationalITy" type ="text" value="KW" /> </p>

92.         <p> Select PlaceofBirth :
93.         <select name="PlaceofBirth">
94.             <option value="KuwaIT">KuwaIT </option>
95.             <option value="Egypt">Egypt </option>
96.             <option value="USA">USA</option>
97.             <option value="lebanon">lebanon</option>
98.             <option value="Syria" selected>Syria </option>
99.         </select>
100.            <p> Select StageID:<input name="StageID" type ="text" value="A" /> </p>
101.            <p> Select GradeID: <input name="GradeID" type ="text" value="G-04" /> </p>
102.            <p> Select SectionID: <input name="SectionID" type ="text" value="A" /> </p>
103.            <p> Select Topic: <input name="Topic" type ="text" value="IT" /> </p>
104.            <p> Select Semester: <input name="Semester" type ="text" value="F" /> </p>
105.            <p> Select Relation: <input name="Relation" type ="text" value="Father" /> </p>
106.            <p> Select raisedhands: <input name="raisedhands" type ="number" value="15" />
    </p>
107.            <p> Select VisITedResources: <input name="VisITedResources" type ="number" v
    alue="25" /> </p>
108.            <p> Select AnnouncementsView: <input name="AnnouncementsView" type ="nu
    mber" value="0" /> </p>
109.            <p> Select Discussion: <input name="Discussion" type ="number" value="70" /> <
    /p>
110.            <p> Select ParentAnsweringSurvey: <input name="ParentAnsweringSurvey" type
    ="text" value="Yes" /> </p>
111.            <p> Select ParentschoolSatisfaction: <input name="ParentschoolSatisfaction" type
    ="text" value="Good" /> </p>
112.            <p> Select StudentAbsenceDays: <input name="StudentAbsenceDays" type ="text
    " value="Under-7" /> </p>
113.
114.
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms

```
                <p> <input type ="submit" value="submit" /> </p>
    </form>
    </body>
    </html>
    <!DOCTYPE html>
    <html>
    <title>Student-Performance-Prediction App</title>
    <body>
    <h3> Student Prediction: {{ labels5 }} </h3>
    <p><h4> [L]  </h4></p>
    <p><h4> [M]  </h4></p>
    <p><h4> [H]  </H4></P>
    </body>
    </html>
```

This content downloaded from
103.176.152.28 on Mon, 29 Sep 2025 15:22:02 UTC
All use subject to https://about.jstor.org/terms